

Sistema de E-commerce com Microsserviços

Relatório Técnico

1. Como a comunicação entre os microsserviços foi implementada?

A comunicação entre os microsserviços foi implementada via HTTP/REST com troca de mensagens no formato JSON. Cada serviço expõe uma API FastAPI independente, rodando em sua própria porta. O API Gateway atua como ponto de entrada único e repassa as requisições aos serviços internos utilizando a biblioteca `httpx` com chamadas assíncronas. O Serviço de Pedidos também realiza chamadas HTTP internas ao Serviço de Usuários e ao Serviço de Produtos para validar a existência dos recursos antes de criar um pedido. A autenticação é propagada pelo gateway via repasse do header `Authorization` original, permitindo que cada serviço valide o JWT de forma independente.

2. Qual estratégia de consistência foi adotada na replicação? Forte ou eventual? Por quê?

Foi adotada consistência forte e síncrona. Quando um produto é criado na réplica primária (porta 5002), a escrita é primeiro propagada para a réplica secundária (porta 5012) via chamada HTTP ao endpoint interno `/_replicate`. Somente após a réplica secundária confirmar o recebimento é que a réplica primária salva o dado localmente e responde ao cliente. Caso a réplica secundária esteja indisponível, a operação de escrita é cancelada e um erro 503 é retornado, garantindo que as réplicas nunca fiquem com dados divergentes. As leituras são distribuídas entre as réplicas por round-robin simples implementado no API Gateway — a cada requisição GET, o gateway alterna entre a primária (5002) e a secundária (5012), com fallback automático caso uma delas esteja indisponível. A escolha pela consistência forte foi feita pela simplicidade de implementação e pela garantia de que qualquer réplica pode responder leituras com dados sempre atualizados.

3. O que acontece com o sistema se o Serviço de Pedidos cair?

O sistema continua funcionando parcialmente. Os Serviços de Usuários e de Produtos permanecem operacionais e independentes, pois não dependem do Serviço de Pedidos. O API Gateway detecta a falha via heartbeat após no máximo duas verificações consecutivas sem resposta (aproximadamente 10 segundos). A partir desse momento, qualquer requisição direcionada ao Serviço de Pedidos recebe uma resposta 503 `Service Unavailable` imediatamente, sem que o gateway tente fazer a chamada. O evento de falha e a posterior recuperação são registrados em log com timestamp. Quando o serviço voltar a responder ao health check, o gateway retoma o roteamento normalmente de forma automática.

4. Como o JWT garante que um usuário comum não consiga criar produtos?

No momento do login ou registro, o Serviço de Usuários gera um token JWT assinado com a chave secreta definida na variável `JWT_SECRET`. Esse token contém o campo `role`, que pode ser `'user'` ou `'admin'`. O endpoint `POST /products` do Serviço de Produtos utiliza uma dependência chamada `require_admin`, que decodifica e valida o token recebido no header `Authorization`. Se o token for válido mas o campo `role` não for `'admin'`, a requisição é rejeitada com erro 403 `Forbidden` antes de

qualquer processamento. Como o token é assinado digitalmente, um usuário comum não consegue alterar o campo role sem invalidar a assinatura, tornando impossível forjar permissões de administrador sem conhecer a chave secreta.

5. Quais limitações a sua implementação possui em relação a um sistema real de produção?

A implementação possui as seguintes limitações em relação a um sistema de produção:

- Armazenamento em arquivos JSON: não há suporte a acesso concorrente seguro entre múltiplos processos ou workers. Em produção, seria necessário um banco de dados relacional ou NoSQL com controle de transações.
 - Hash de senhas com SHA-256 puro: sem salt, o que torna as senhas vulneráveis a ataques de dicionário e rainbow tables. Em produção, deve-se usar bcrypt, argon2 ou scrypt.
 - Replicação síncrona sem tolerância a falhas parciais: se a réplica cair, escritas são bloqueadas. Um sistema real usaria filas de mensagens (ex: Kafka, RabbitMQ) para replicação assíncrona com resiliência.
 - Sem controle de estoque transacional: a verificação e o decremento de estoque no momento do pedido não são atômicos, o que pode causar problemas de concorrência (overselling) em alto volume.
 - Sem HTTPS: toda comunicação é em texto puro. Em produção, todos os endpoints devem usar TLS.
 - Chave JWT única e sem rotação: em produção, deve-se implementar rotação de chaves e suporte a refresh tokens.
 - Sem paginação: endpoints como GET /products e GET /orders retornam todos os registros, o que não escala.
-